

บทที่ 2

การเขียนโปรแกรมติดต่อ ระหว่างแอปพลิเคชัน กับวินโดวส์โดยใช้ API

แม้ว่าภาษาวิซวลเบสิก จะเตรียมฟังก์ชัน และความสามารถต่างๆ ไว้อย่างมากมาย แต่อาจไม่ตอบสนองความต้องการทั้งหมด ดังนั้นวิซวลเบสิกจึงยอมให้ผู้พัฒนาแอปพลิเคชัน สามารถที่จะเรียกใช้ความสามารถ ของระบบปฏิบัติการทั้ง Windows 95/98 ,NT Workstation ได้อย่างเต็มที่ โดยการเรียกผ่านฟังก์ชัน Windows API

2.1 API คือ

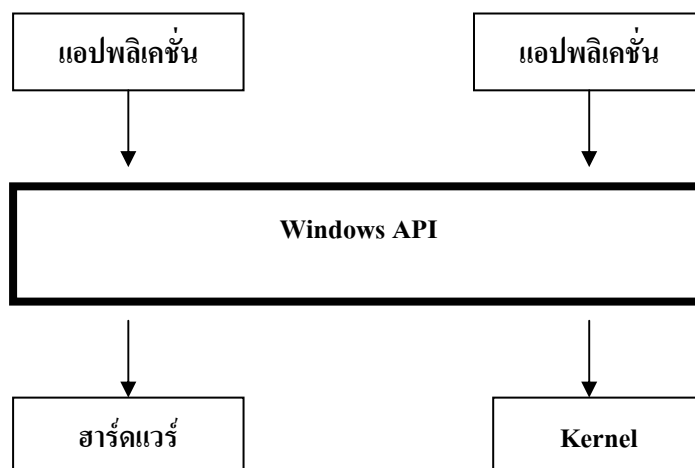
API ย่อมาจาก (Application Programming Interface) ซึ่งเป็นส่วนเชื่อมต่อระหว่างแอปพลิเคชัน กับ วินโดวส์โดยจะแสดงลักษณะการติดต่อสื่อสารกันระหว่างแอปพลิเคชัน กับวินโดวส์ ทำให้เข้าใจได้ว่าแอปพลิเคชัน กับวินโดวส์มีการติดต่อสื่อสารกันอย่างไร โดยมีลักษณะการทำงานเป็นแบบ Event-Driven Programming ซึ่งมีลักษณะการทำงานที่ทำให้ผู้ใช้เป็นอิสระแตกต่างจากแบบเดิมที่เป็น Sequence Programming และเป็นแบบ Graphic Oriented โดยจะมีมีฟังก์ชันอยู่มากมายนับพันๆ ฟังก์ชัน ถูกแบ่งออกเป็นหมวดหมู่ตามประเภทของงาน เช่น API ด้านกราฟฟิก , API ด้านเน็ตเวิร์ค เป็นต้นเรา

สำหรับสาเหตุที่ต้องมี Windows API ก็เพราะว่า เราจะได้มีวิธีการมาตรฐานในการใช้งาน วินโดวส์แบบเดียว ซึ่งจะช่วยลดเวลาของเราที่ต้องไปศึกษาอุปกรณ์ และองค์ประกอบต่างๆ ทำให้เราไม่จำเป็นต้องรู้ว่าจอภาพทำงานอย่างไร, แป้นพิมพ์ทำงานอย่างไร เม้าส์ทำงานอย่างไร แต่สามารถเรียกใช้งานผ่าน Windows API ได้ทันที

อีกหนึ่งข้อดีของ Windows API ที่น่าสนใจก็คือ ช่วยลดความผิดพลาดจากการเข้าถึงฮาร์ดแวร์โดยตรง เพราะเราจะสั่งงานฮาร์ดแวร์ผ่าน Windows API เท่านั้น ซึ่งช่วยลดความสับสนในการใช้งานฮาร์ดแวร์ที่ผิดวิธีไปได้มาก

นอกจากนี้ยังมีข้อดีของ Windows API ได้แก่ ความเป็นมาตรฐานที่ทุกคนรู้จัก จึงง่ายต่อการแก้ไขเปลี่ยนแปลง เช่น แม้ว่าเราจะเปลี่ยนไปใช้ฮาร์ดแวร์ใหม่ หรือแม้แต่วิธีการเวอร์ชันใหม่ๆ แล้วก็ตาม ชื่อของฟังก์ชันเหล่านี้ ยังคงใช้ชื่อเดิมทำงานได้เหมือนเดิม ทำให้เราไม่ต้องกังวลกับความเปลี่ยนแปลงที่จะเกิดขึ้นจนบ่อยเกินไปนัก

ซึ่งมันก็คือ ชุดของฟังก์ชันที่พร้อมให้เราเรียกใช้งาน โดยที่ฟังก์ชันเหล่านั้นรับประกันได้ว่าทำงานอย่างมีประสิทธิภาพ เช่น เราไม่จำเป็นต้องเขียนโปรแกรมให้ยืดยาวเพื่อตรวจสอบว่าเนื้อที่ฮาร์ดดิสก์เหลือว่างอยู่เท่าไร แต่เรียกใช้ฟังก์ชัน GetDiskFreeSpace ฟังก์ชันเดียวก็รู้แล้ว เป็นต้น ซึ่งในรูปที่ 2-1 จะแสดงหลักการทำงานของ Windows API



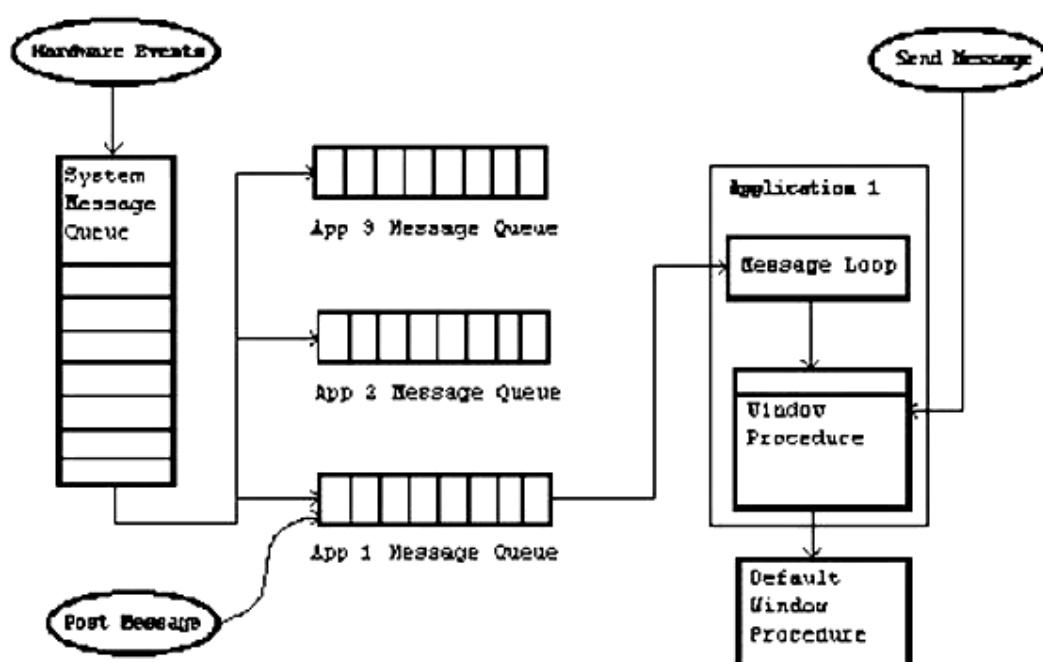
รูปที่ 2-1 แสดงหลักการทำงานของ Windows API

โดยในขั้นต้นเรามาทำความเข้าใจเกี่ยวกับการทำงานแบบ Event-Driven Programming กันก่อน

2.2 ลักษณะของ Event-Driven Programming

มีลักษณะการทำงานที่ไม่เป็นลูกโซ่ที่ต้องทำตามลำดับ โดยโปรแกรมจะทำงานตามเหตุการณ์ที่ผู้ใช้กระทำ โดยที่โปรแกรมจะไม่ใช่ตัวชักจูงทำตามผู้ใช้ให้ทำตามทีโปรแกรมตั้งไว้ โดยที่ระบบปฏิบัติการจะเป็นผู้ตรวจสอบเหตุการณ์ที่เกิดขึ้นแล้วส่งข้อมูลของเหตุการณ์นั้น(Message) ไปให้ยังโปรแกรม โดยที่โปรแกรมจะทำงานตามข้อมูลของเหตุการณ์นั้น ดังจะแสดงให้เห็นในรูปที่ 2-2

Windows Events and Messages



รูปที่ 2-2 แสดงการทำงานของ API Programming โดยทำงานแบบ Event-Driven Programming

2.3 ลักษณะโครงสร้างการทำงานของ Windows API

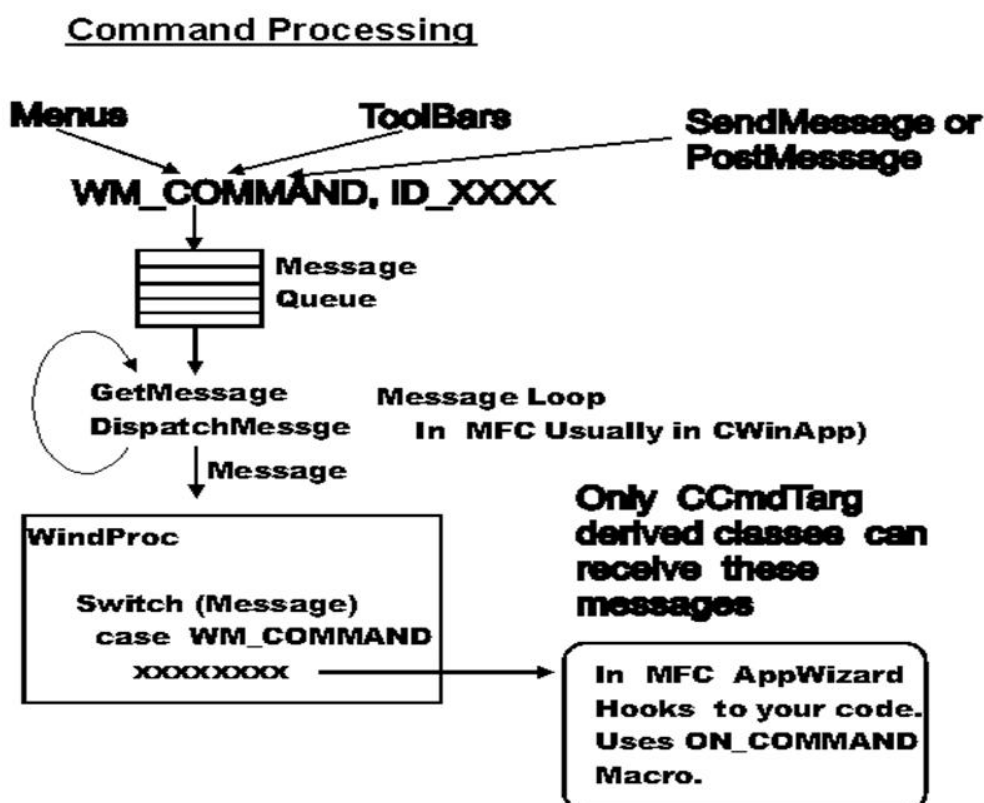
ลักษณะการทำงานของ Windows API จะมีขั้นตอนคร่าวๆดังนี้

1. กำหนดค่าตัวแปรเริ่มต้นและเนื้อที่ของหน่วยความจำ (Initial Variable and Memory Space)
2. สร้างและแสดงหน้าต่างของโปรแกรม
3. จะเข้า Loop การทำงาน เพื่อรอรับ Message ของทำงาน
 - เพื่อรอรับ Message การทำงานที่ส่งมาจากระบบปฏิบัติการมายัง แอปพลิเคชันของเรา
 - ถ้า Message คือ WM_QUIT จะจบการทำงานและคืนค่าการควบคุมกลับไปยัง

ระบบปฏิบัติการ

- ทำ Message เป็นคำสั่งที่มาจากผู้ใช้ ก็จะทำตามข้อมูลที่ส่งมานั้นๆ

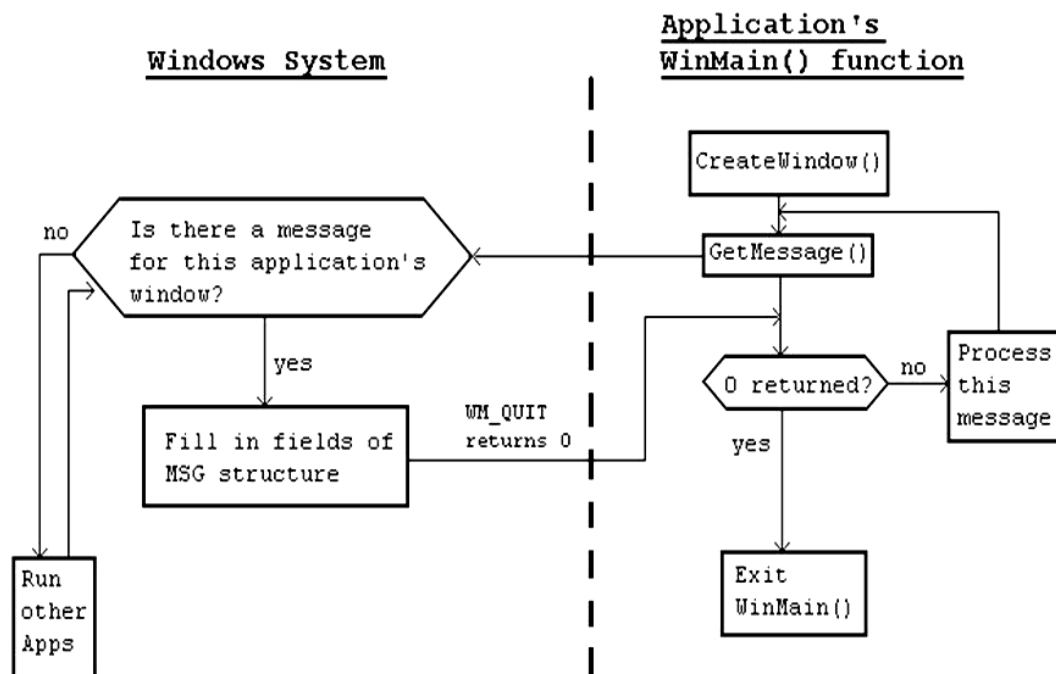
โดยขั้นตอนการทำงานจะแสดงในรูปที่ 2-3 และ 2-4



รูปที่ 2-3 แสดงการลำดับขั้นตอนการทำงานของ Windows API Programming

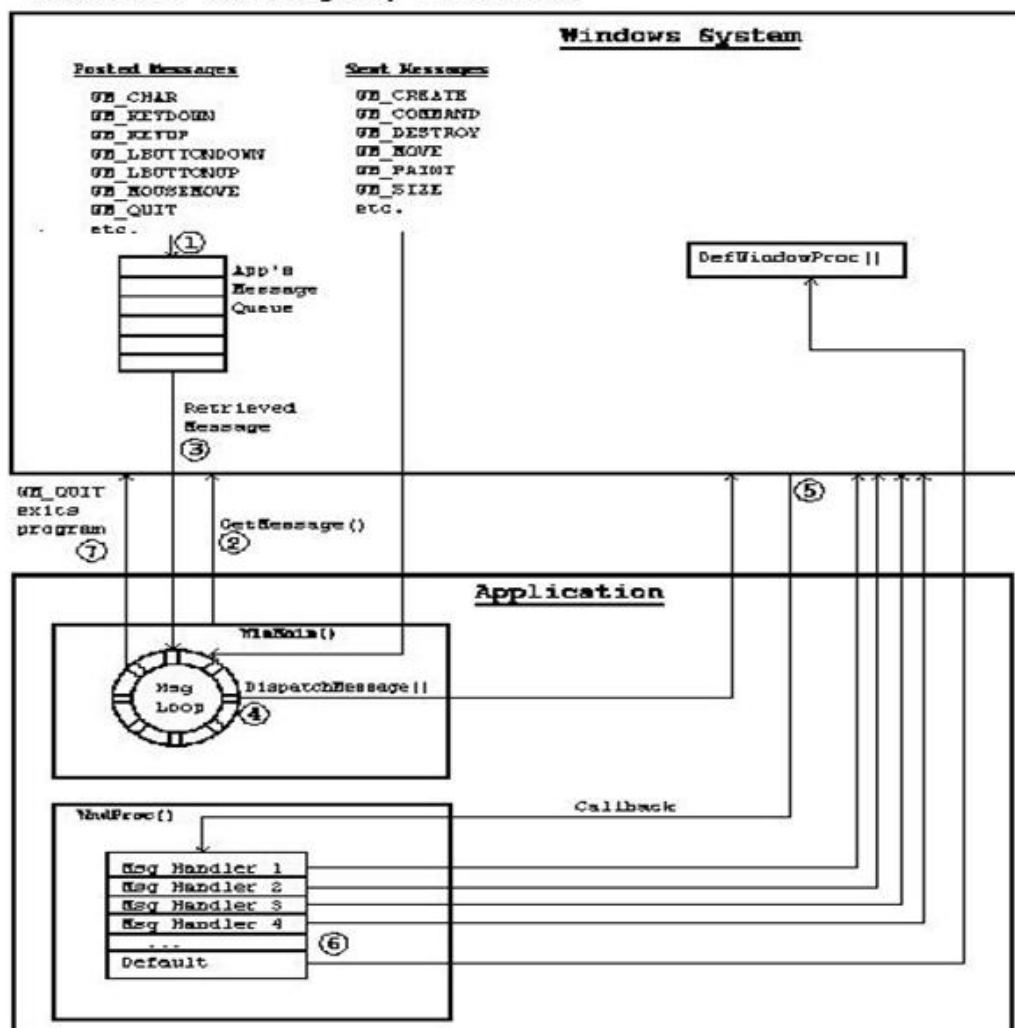
โดยรูปที่ 2-4 จะแสดงขั้นตอนการทำงานของโปรแกรมจนจบการทำงาน โดยจะจบการทำงานเมื่อได้รับ Message “WM_QUIT”

The Main Message Loop



รูปที่ 2-4 แสดงขั้นตอนการทำงานของ Windows API Programming ตั้งแต่เริ่มต้นจนจบการทำงาน

Windows Messages, Details



รูปที่ 2-5 แสดงการทำงานของ Message ชนิดต่างๆ

ในหัวข้อนี้เราจะเรียนรู้วิธีการเรียกใช้ฟังก์ชัน API พร้อมทั้งบอกถึงฟังก์ชันที่ถูกนำมาใช้งาน

2.4 กลุ่มฟังก์ชันของ Windows API

Windows 95/98, NT Workstations จะมีฟังก์ชันในรูปของ Windows API ให้ใช้มากกว่า 1000 ฟังก์ชัน ซึ่งฟังก์ชันต่างๆ จะถูกเก็บในไฟล์ .DLL ซึ่งอยู่ในโฟลเดอร์ C:\Windows\System ซึ่งไฟล์สำคัญๆ ได้แก่

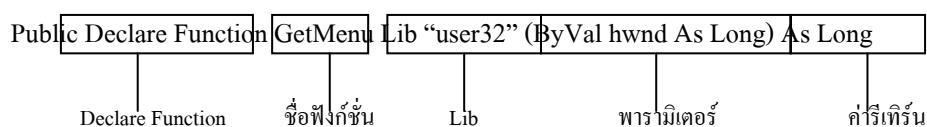
ชื่อไฟล์	คำอธิบาย
ADVAPI32.DLL	มาจาก Advanced API ซึ่งจะเก็บฟังก์ชันเกี่ยวกับความปลอดภัย และ Registry
COMDLG.DLL	เก็บฟังก์ชันของ Common Dialog
GDI32.DLL	GDI ซึ่งจะเก็บฟังก์ชันเกี่ยวกับการแสดงผล และฟังก์ชันเกี่ยวกับกราฟฟิก
KERNEL32.DLL	เก็บฟังก์ชันที่ใช้จัดการทรัพยากรของระบบปฏิบัติการ
LZ32.DLL	เก็บฟังก์ชันที่ใช้เกี่ยวกับการบีบอัดข้อมูล
MPR.DLL	เก็บฟังก์ชันที่เกี่ยวข้องกับ Multiple Provide Router
NETAPI32.DLL	เก็บฟังก์ชันที่เกี่ยวข้องกับการจัดการเครือข่าย (Network)
SHELL32.DLL	เก็บฟังก์ชันที่เกี่ยวข้องกับเชลล์ (Windows Explorer)
USER32.DLL	เก็บฟังก์ชันที่เกี่ยวข้องกับการติดต่อกับผู้ใช้งาน (User Interface)
VERSION32.DLL	เก็บฟังก์ชันที่เกี่ยวข้องกับการจัดการรุ่นของซอฟต์แวร์
WINMM.DLL	เก็บฟังก์ชันที่เกี่ยวข้องกับมัลติมีเดีย
WINSPOOL.DRV	เก็บฟังก์ชันที่เกี่ยวข้องกับการพิมพ์ผ่านเครื่องพิมพ์

ตารางที่ 2-1 แสดงตารางเกี่ยวกับไฟล์ DLL ที่สำคัญ

2.5 การประกาศฟังก์ชันใน Windows API

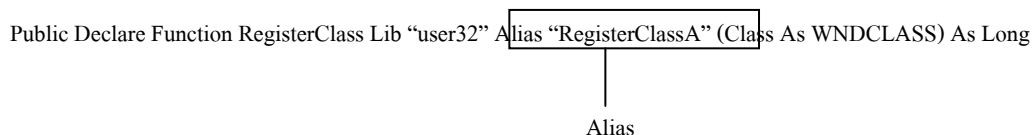
ในการเรียกใช้ฟังก์ชันต่างๆ ของ Windows API นั้น มีขั้นตอนที่แตกต่างจากการใช้เรียกฟังก์ชันที่วิซวลเบสิกเตรียมไว้ให้ หรือฟังก์ชันที่เราสร้างขึ้นมานั่นคือ ต้องมีการประกาศค่าฟังก์ชันก่อนการใช้งาน

สำหรับการใช้งานฟังก์ชันนั้น เราจะคัดลอกมาจาก API Viewer ซึ่งฟังก์ชันที่คัดลอกมานั้น มีรูปแบบการประกาศดังนี้



จากตัวอย่าง เป็นการประกาศฟังก์ชัน GetMenu และฟังก์ชัน RegCloseKey จะเห็นว่าฟังก์ชันที่ประกาศนั้น ประกอบด้วยองค์ประกอบต่างๆ ดังนี้

- Declare Function เป็นคำสั่งแสดงว่านี่คือ การประกาศฟังก์ชัน
- ชื่อฟังก์ชัน เป็นชื่อของฟังก์ชัน จากตัวอย่างนี้ ชื่อของฟังก์ชัน คือ GetMenu และ RegClose
- Lib เป็นการบอกให้ทราบว่าให้ค้นหาจากไฟล์ .DLL ซึ่งจะต้องระบุชื่อไฟล์ .DLL นั้น ยกเว้นอยู่ในไฟล์ USER32.DLL, KERNEL32.DLL และ GDI32.DLL
- พารามิเตอร์ เป็นรายการของพารามิเตอร์ที่ต้องมีการผ่านค่าให้ฟังก์ชัน
- ค่าที่รี턴 เป็นชนิดข้อมูลที่จะคืนค่าให้กับผู้เรียกใช้ จากตัวอย่างมีชนิดข้อมูลเป็น Long ในบางฟังก์ชันนั้น อาจมีองค์ประกอบอื่นเพิ่มเติม เช่น



- Alias เป็นชื่อที่ระบบปฏิบัติการเรียกใช้ฟังก์ชันนี้ จากตัวอย่างเราจะเรียกใช้งานในแอปพลิเคชันของเราในชื่อฟังก์ชัน RegisterClass แต่ Windows 95/98 จะรู้จักฟังก์ชันเดียวกันนี้ในชื่อ RegisterClassA เป็นต้น